The pxLST package

Professional code listing environments for LaTeX2e

Scvria
P3CTeX / UOC PEC Repository

2026/03/04 – Version 0.1

Abstract

pxLST provides a high-level LaTeX2e API for typesetting syntax-highlighted source code listings inside styled tcolorbox frames. It wraps the listings and tcolorbox packages behind a unified key–value interface (the pxLST key family) and ships four visual style renderers based on the Darcula colour palette, twelve pre-defined language grammars (Java, Prolog, OWL, TAD, YAML, Catalan pseudocode, Bash, Python, C, JavaScript, SQL), a named preset system, and a diagnostic probe for quality-assurance harnesses. Full UTF-8 literate support for accented Latin characters is included at the global lstset baseline.

Contents

1 Loading the package

```
\usepackage{pxLST}
```

Package options are forwarded to the internal key family `pxLST` and are equivalent to calling `\pxLSTsetup` in the preamble. All options are `key=value` pairs:

```
\usepackage[style=light,language=pxBash]{pxLST}
```

Defaults are chosen to be sensible for course work and technical reports: dark Darcula background, Java language, line numbers on the left. All keys affect subsequent listings until changed again; the usual LaTeX group scoping rules apply.

2 Key reference

The `pxLST` key family contains the following keys. All of them can be set at load time via `\usepackage[<keys>]{pxLST}`, globally at any point via `\pxLSTsetup{<keys>}`, or locally per listing in the optional `[<options>]` argument of every environment and command.

Key	Type	Default	Description
`style`	choice	`dark`	Visual renderer: `dark`, `framed`, `light`, `plain`.
`language`	tl	`pxJava`	Any `\lstdefinelanguage` name; controls grammar and k
`numbers`	tl	`left`	Line number position: `left`, `right`, `none`.
`caption`	tl	(empty)	Caption text; stored but not auto-used in `pxCode`.
`label`	tl	(empty)	Cross-reference label; inserted as `\label{lst:<value>}`.
`float`	tl	(empty)	Float placement (`h`, `t`, `htbp`...); empty = inline.
`breakable`	bool	`true`	Allow `tcolorbox` page-break; `pxCodeBreak` forces true.
`fontsize`	tl	`\footnotesize`	Font-size command applied in `basicstyle`.
`linespread`	tl	`1.2`	Numeric factor forwarded to `\linespread` inside `basicst`
`tabsize`	int	`4`	Tab width in spaces (forwarded to `listings`).
`preset`	code	—	Apply a named preset; calls `\pxLSTusepreset{<name>}`.
`probe`	bool	`false`	Emit `PXLST_ASSERT:` diagnostics to the log; see §11.

Unknown keys are silently ignored (reserved for future extension).

Key interaction notes.

- `style` controls both the listings colour scheme (`\lstdefinestyle`) and the tcolorbox frame colours (`colback`, `colframe`, `coltext`).
- `language` is independent of `style`; both apply simultaneously to the same listing box.
- `preset` is a `.code:n` key, so subsequent keys override preset values when they appear after `preset` in the option list. See §9.
- `float` and `breakable` are orthogonal: if `float` is non-empty the listing is wrapped in a `figure` float, and `breakable` is ignored (floats cannot break across pages in standard LaTeX).
- `caption` and `label` are stored but are *not* automatically used in `pxCode`; they are used by `pxCodeBox` and `pxCodeBreak` to build the title bar and place `\label`.

3 Global configuration — \pxLSTsetup

`\pxLSTsetup{<key>=<value>,...}`

Sets package-wide defaults in the `pxLST` key family. Usual LaTeX scoping rules apply, so a setup call inside a group affects only the listings in that group. At document level the settings remain in effect until overridden by another `\pxLSTsetup` call or until a surrounding group ends.

Example — select the light style and Bash language for all subsequent listings in the current scope:

```
\pxLSTsetup{style=light, language=pxBash, numbers=none}
```

4 Environments

All three environments below use the same `pxlst@configure` mechanism internally: the optional `[<options>]` argument is processed first (applying local key overrides), then the display macros are updated inside a TEX group, so settings revert automatically when the environment ends.

4.1 pxCode

```
\begin{pxCode}[<options>]{<label>}
  ... code content ...
\end{pxCode}
```

- `#1` (optional) — key–value overrides, local to this environment instance.
- `#2` (mandatory) — label string; displayed in the title bar as `\texttt{\bfseries #2}` and used as the LATEX label anchor `\label{lst:<label>}` when non-empty.

The environment wraps a `listings lstlisting` environment inside a `tcolorbox` with `listing only` mode, a `drop lifted shadow` decoration, and a rounded frame (`arc=.25em`). It is defined via `\newtcblisting` so that verbatim code inside is handled correctly.

Example:

```
\begin{pxCode}[language=pxJava, numbers=left]{HelloWorld.java}
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, world!");
    }
}
\end{pxCode}
```

4.2 pxCodeBox

```
\begin{pxCodeBox}[<options>]{<label>}{<title>}{<note>}{<caption>}
  ... code content ...
\end{pxCodeBox}
```

- `#1` (optional) — key–value overrides, local scope.
- `#2` — label string; inserted as `\label{cdbx:<label>}` in the title bar and used in continuation headers on subsequent pages.
- `#3` — title text, shown right-aligned in the title bar.

- `#4` — short note identifier, shown left-aligned in the title bar in a smaller, lighter font.
- `#5` — caption text, rendered in the comment panel below the listing.

`pxCodeBox` renders a two-region `tcolorbox`: the main (top) panel contains the syntax-highlighted listing; the comment (bottom) panel displays the `#5` caption alongside a small codebox anchor label derived from `#2`. The box is *not* breakable by default (use `pxCodeBreak` for long listings).

Example:

```
\begin{pxCodeBox}[style=dark, language=pxJava]
                 {alg:bubbleSort}
                 {BubbleSort.java}
                 {alg}
                 {Classic O($n^2$) comparison sort.}
public void bubbleSort(int[] a) {
    for (int i = 0; i < a.length - 1; i++)
        for (int j = 0; j < a.length - 1 - i; j++)
            if (a[j] > a[j+1]) { int t = a[j]; a[j]=a[j+1]; a[j+1]=t; }
}
\end{pxCodeBox}
```

4.3 pxCodeBreak

```
\begin{pxCodeBreak}[<options>]{<label>}{<title>}{<note>}{<caption>}
  ... code content ...
\end{pxCodeBreak}
```

Identical signature to `pxCodeBox` but forces `breakable=true` regardless of any key setting. Use for long listings that span multiple pages; the title bar is repeated on continuation pages with a "…continues from page N" prefix.

5 File input — \pxLSTinput

```
\pxLSTinput[<options>]{<filename>}
```

Reads an external source file and typesets it as a listing using the current key settings (with optional local overrides in `#1`). Internally delegates to a `tcolorbox` that wraps `\lstinputlisting`, so the full Darcula frame is preserved.

- `#1` (optional) — any `pxLST` key–value pairs.

- `#2` — filename, relative to the document's current working directory (i.e. where `pdflatex` is invoked from).

Example:

```
\pxLSTinput[style=light, language=pxBash]{scripts/build.sh}
```

Unlike the verbatim environments, `\pxLSTinput` is defined via `\NewDocumentCommand` and can therefore appear in moving arguments and macros without special treatment.

6 Inline code

`\pxCodeInline` typesets a single short code snippet *inline* in the main text, inside a small tcolorbox with syntax highlighting.

Naming: pxCode environment vs. inline code command.

- **pxCode environment** — `\begin{pxCode}[<options>]{<label>}` ... `\end{pxCode}`. A single block of verbatim code inside a styled tcolorbox; see §7.
- **Inline code command** — `\pxCodeInline`. Takes one snippet per call; renders it inline in a small box. Content is normal braced material (not verbatim). For multiple snippets use multiple `\pxCodeInline` calls. For long verbatim blocks use the **pxCode environment**.

6.1 `\pxCodeInline`

```
\pxCodeInline[<options>]{<snippet>}
```

- `#1` (optional) — `pxLST` keys: `style`, `language`, `preset`, `fontsize`, `linespread`, `tabsize`. Keys `numbers`, `caption`, `label`, `float`, `breakable` are ignored for inline.
- `#2` (mandatory) — one code snippet.

Content is *not* verbatim: special characters (`#`, `%`, `&`) must be escaped as in normal LaTeX. For full verbatim blocks use `\begin{pxCode}...\end{pxCode}`.

Example.

```
Use \pxCodeInline{int x = 0;} and \pxCodeInline{return x + 1;} in your method.
\pxCodeInline[language=pxBash]{echo "done"}
```

`\pxLSTsetup` and presets apply: e.g. `\pxLSTsetup{style=light}` affects the style used for `\pxCodeInline`.

7 Style reference

pxLST provides four visual style renderers, each implemented as a `\lstdefinestyle` entry paired with corresponding `tcolorbox` colour parameters. All four styles use the Darcula colour palette defined at package load time.

Style	Background	Frame	Syntax colours
`dark`	Darcula dark (`RGB{40,44,52}`)	none	Full Darcula palette on d
`framed`	Darcula dark	left rule (darcula-comment)	Full Darcula palette; left
`light`	white	none	Darcula tones darkened f
`plain`	white	none	Monochrome: keywords b

tcolorbox parameters per style.

Style	colback	colframe	coltext	arc
`dark`	`darcula-bg!90!`	`darcula-bg!60!`	`darcula-fg`	`.25em`
`framed`	`darcula-bg!90!`	`darcula-comment!60!`	`darcula-fg`	`.25em`
`light`	`white`	`gray!25!`	`black`	`.25em`
`plain`	`white`	`gray!15!`	`black`	`0pt`

Switching styles.

```
% Globally:
\pxLSTsetup{style=framed}

% Locally in one listing:
\begin{pxCode}[style=light]{MyFile.java}
  ...
\end{pxCode}
```

The `dark` and `framed` styles are recommended for standalone course submissions and printable reports where a clear visual boundary is desired. The `light` style suits documents that mix code with running prose and share the page background with text blocks. The `plain` style is appropriate for black-and-white printing where colour ink should be avoided.

8 Language reference

Twelve language grammars are defined in `pxLST.code.tex` via `\lstdefinelanguage`. They are available regardless of the active style.

Language key	Based on	Typical use
`pxJava`	Java	Java source, annotations (`@`), generics.
`pxProlog`	Prolog	Logic programming, facts, rules, DCG clauses.
`pxOWL`	—	Manchester OWL ontology syntax.
`pxTAD`	—	Abstract data type (TAD) specifications.
`pxYAML`	—	YAML configuration files.
`algoritme`	—	Catalan pseudocode with rendered math symbols.
`pxBash`	bash	Shell scripts, command-line listings.
`pxPython`	Python	Python source.
`pxC`	C	C source.
`pxJavaScript`	—	JavaScript/ECMAScript.
`pxJS`	pxJavaScript	Alias for `pxJavaScript`.
`pxSQL`	—	SQL.

UTF-8 literate map. The global `\lstset` baseline loaded by `pxLST` includes a comprehensive UTF-8 literate mapping for accented characters: acute, grave, umlaut, and circumflex accents are mapped for all letters `a-z` and `A-Z`, plus the digraphs `ç`, `Ç`, `ñ`, `Ñ`, `¿`, and `¡`. This means Catalan, Spanish, and French source files containing accented identifiers or comments are typeset correctly without any manual literate entry.

The `algoritme` language. This language is designed for Catalan pseudocode used in UOC coursework. It maps common ASCII sequences to proper mathematical symbols:

Input	Output
`<--` (three chars)	← (assignment arrow)
`<=`	≤
`>=`	≥
`!=`	≠
`*`	×

Keywords (`Funcio`, `Fi`, `Si`, `Llavors`, `Sinó`, `Mentre`, `Per`, `Retornar`…) are rendered bold in the Darcula keyword colour; case-sensitive matching is active.

9 Preset system

A *preset* is a named bundle of key–value settings that can be saved once and applied to any number of listings. Presets survive group boundaries when saved, but their *effect* is local to the scope in which they are applied.

9.1 Preset commands

- `\pxLSTsavepreset{<name>}{<option-string>}`
  Stores the option string under the given name. If a preset with the same name already exists it is silently overwritten.

- `\pxLSTusepreset{<name>}`
  Applies the preset in the current scope, equivalent to calling `\pxLSTsetup` with the stored option string. If the preset is not found, a warning is written to the log and the call is ignored.

- `preset=<name>` key
  The `preset` key can be used inside any option list. Because keys are processed in source order, keys appearing *after* `preset` override preset values:

```
% 'darcula' sets language=pxJava; the subsequent key overrides it.
\pxLSTsetup{preset=darcula, language=pxBash}
```

9.2 Built-in presets

Three generic presets are always available (registered at package load time):

Preset	Key bundle
`darcula`	`style=dark, language=pxJava, numbers=left, breakable=true`
`minimal`	`style=plain, language=pxJava, numbers=none`
`academic`	`style=light, language=pxJava, numbers=left, fontsize=\small`

9.3 P3CTeX branded presets

When `pxLST` is loaded via the `P3CTeX` document class, three additional branded presets are registered automatically before option processing, so they can be used in `\usepackage` options:

```
\usepackage[preset=p3c-code]{pxLST}
```

Preset	Key bundle
`p3c-code`	`style=dark, language=pxJava, numbers=left, breakable=true`
`p3c-alg`	`style=framed, language=algoritme, numbers=left, breakable=false`
`p3c-plain`	`style=plain, language=pxJava, numbers=none, breakable=false`

9.4 Usage example

```
% Define a custom preset:
\pxLSTsavepreset{myprolog}{style=framed, language=pxProlog, numbers=none}

% Apply it globally:
\pxLSTusepreset{myprolog}

% Or locally:
\begin{pxCode}[preset=myprolog]{family.pl}
  parent(tom, bob). parent(tom, liz).
  ancestor(X,Y) :- parent(X,Y).
\end{pxCode}

% Override one key after preset:
\begin{pxCode}[preset=myprolog, numbers=left]{rules.pl}
  ...
\end{pxCode}
```

10 pxCORE integration

`pxLST` is an opt-in module of `pxCORE`, the P3CTeX core package. By default, `pxCORE` does *not* load `pxLST` (initial value of the `LST` module key is `false`).

Explicit opt-in.

```
\usepackage[LST]{pxCORE}
```

This sets `LST=true` and causes `\RequirePackage{pxLST}` to be executed during `pxCORE` loading.

Bundle (all modules).

```
\usepackage[default]{pxCORE}
```

The `default` meta-key activates all modules: `PRP`, `UML`, `SRC`, `TAB`, and `LST`. This is the recommended setup when using the full P3CTeX workflow.

Module key summary.

Usage	LST loaded?	Note
`\usepackage{pxCORE}`	No	Default (opt-in required).
`\usepackage[LST]{pxCORE}`	Yes	Explicit opt-in.
`\usepackage[default]{pxCORE}`	Yes	All modules bundle.

11 Diagnostic probe

`\pxLSTsetup{probe=true}`

When `probe` is set to `true`, every call to `\pxLSTsetup` (and every environment invocation) writes a block of diagnostic lines to the LaTeX log via `\iow_term:x`. These lines start with the prefix `PXLST_ASSERT:` and are intended for automated QA harnesses that parse the log file.

Asserted values.

Log key	Source variable
`PXLST_ASSERT: STYLE`	`\l_pxlst_style_tl` (e.g. `dark`)
`PXLST_ASSERT: LANGUAGE`	`\l_pxlst_language_tl` (e.g. `pxJava`)
`PXLST_ASSERT: NUMBERS`	`\l_pxlst_numbers_tl` (e.g. `left`)
`PXLST_ASSERT: COLBACK`	`\pxlst@colback@v` (e.g. `darcula-bg!90!`)
`PXLST_ASSERT: LSTYLE`	`\pxlst@lstyle@v` (e.g. `pxlst@dark`)

Probe output has no effect on the typeset document. Disable it again with `\pxLSTsetup{probe=false}` once debugging is complete.

12 Known limitations (v0.1)

The following items are explicitly outside the scope of the current release and are deferred to a future sprint.

- **No `minted` backend.** `pxLST` v0.1 uses the `listings` package exclusively. A Pygments-based (`minted`) rendering backend is a v0.2 candidate.
- **No dedicated listing float counter.** The `float` key wraps listings in a `figure` environment. A separate `listing` float counter (compatible with `\listoflistings`) is deferred.
- **No automatic language detection.** `\pxLSTinput` does not infer the language from the file extension; the `language` key must be set explicitly.
- **No side-by-side listing environments.** Two-column code comparisons (source vs. output, before vs. after) are not provided in this version.
- **No per-environment keyword extension.** There is no high-level key to add `morekeywords` at use time beyond the fixed language definition; users who need additional keywords must use raw `\lstset` calls.

- **No code annex or list-of-listings.** pxLST focuses on code environments, file input, and inline code only. Any future annex or list-of-listings functionality will live in a dedicated package (e.g. pxANX), not in pxLST.

13 Implementation notes

This section gives a brief account of the internal architecture for maintainers and advanced users.

File organisation

`pxLST` is split across two files:

- `tex/latex/pxLST.sty` — public LaTeX2e API: `\RequirePackage` block, key-processing bootstrap, public commands (`\pxLSTsetup`, `\pxLSTsavepreset`, `\pxLSTusepreset`, `\pxLSTinput`, `\pxCodeInline`), and the three `\newtcblisting` environment declarations.
- `tex/code/pxLST.code.tex` — expl3 internals: variable declarations, key definitions, Darcula colour palette, display macro initialisations, global `\lstset` baseline, four style definitions, seven language definitions, preset mechanism, probe, and `\pxLST_input_file:nn`.

The `.sty` inputs the `.code.tex` file via `\file_input:n{pxLST.code.tex}`, so both files must be on the `TEXINPUTS` path.

Dynamic option mechanism

Since `\newtcblisting` environments must handle verbatim content, they cannot call `\pxLSTsetup` directly inside their option argument. Instead, a PGF key `/tcb/pxlst@configure` is defined in `.code.tex`:

```
\pgfkeys{/tcb/pxlst@configure/.code n args={1}{
  \pxLST_set_options:n { #1 }
  \__pxlst_set_style_vars:
  \tcbset{
    colback  = \pxlst@colback@v,
    colframe = \pxlst@colframe@v,
    ...
  }
}}
```

Each `\newtcblisting` passes `pxlst@configure={#1}` as its first tcolorbox option, which triggers the key processing chain before the box is opened. The display macros (`\pxlst@*@v`) are set via `\def` and therefore restore automatically when the group closed by `\end{pxCode}` (etc.) unwinds.

Dependency stack

`pxLST.sty` loads the following packages in order: `expl3`, `xparse`, `l3keys2e`, `listings`, `listingsutf8`, `xcolor` (with the `table` option), `tcolorbox`. The tcolorbox libraries `listings`, `listingsutf8`, `breakable`, `xparse`, and `skins` are loaded via `\tcbuselibrary`. The `float` package and `amssymb` are *not* required: float wrapping uses the standard `figure` environment, and math symbols in the `algoritme` language are rendered inline via standard LaTeX math mode.